

ΠΛΗΡΩΣΗ ΤΡΙΓΩΝΩΝ

ΜΠΑΚΙΡΤΖΗΣ ΒΑΣΙΛΕΙΟΣ, 9031

Flat shading

Αρχικά, για κάθε ζεύγος κορυφών και την αντίστοιχη πλευρά που αυτές ορίζουν, αποθηκεύονται στον πίνακα `edges`:

- 1) Η ελάχιστη τεταγμένη των κορυφών, δηλαδή η $y_{\min}$.
- 2) Η μέγιστη τεταγμένη $y_{\max}$.
- 3) Η τετμημένη του σημείου με $y_{\min}$.
- 4) Ο λόγος dx/dy , που εκφράζει τη μεταβολή της τετμημένης για μεταβολή της y κατά 1. Με άλλα λόγια, είναι η αντίστροφη κλίση της ευθείας που ενώνει τις κορυφές που ορίζουν την ακμή, και με τη βοήθεια αυτού του λόγου θα βρίσκουμε αργότερα τη νέα τετμημένη του ενεργού σημείου όταν προχωράμε στο επόμενο scanline.

Για $i = 0, 1, 2$ επιλέγονται δύο διαδοχικές κορυφές από τον πίνακα `vertices`:

`vertices[i] & vertices[(i+1) % 3]`

Ο τρόπος με τον οποίο επιλέγονται τα ζεύγη των κορυφών περιλαμβάνει χρήση του τελεστή modulo, που δίνει το υπόλοιπο της ακέραιας διαίρεσης. Έτσι, για $i = 2$, επιλέγεται το ζεύγος `vertices[2] & vertices[0]`. Στην περίπτωση οριζόντιας ακμής, δηλαδή όταν $y_1 = y_2$, αυτή δεν προστίθεται στον πίνακα `edges` και προχωρούμε στην εξέταση της επόμενης περίπτωσης. Προσθέτουμε έναν έλεγχο για το εάν προκύψουν λιγότερες από 2 έγκυρες ακμές. Στην περίπτωση που αυτό συμβαίνει δε γίνεται πλήρωση.

Έπειτα, ταξινομούμε τις καταχωρήσεις της λίστας `edges` (δηλαδή κάθε τετράδα τιμών) κατά το πρώτο στοιχείο τους ($y_{\min}$) σε αύξουσα σειρά με τη βοήθεια της `lambda function` ως τιμή στην παράμετρο `key` της `sorted` συνάρτησης. Βρίσκουμε την ελάχιστη τιμή από όλα τα $y_{\min}$ και τη μέγιστη από όλα τα $y_{\max}$. Αυτό είναι το εύρος των scanlines που μας ενδιαφέρουν, μιας και μόνο αυτά θα τέμνουν το τρίγωνο.

Αρχικοποιούμε τη λίστα στην οποία θα αποθηκεύουμε τις ενεργές ακμές (`active_edges`) κατά τη σάρωση των scanlines και αποθηκεύουμε τις διαστάσεις του καμβά σε δύο μεταβλητές h (height) και w (width). Αυτές θα χρησιμεύσουν για τον έλεγχο στον χρωματισμό ενός pixel αν αυτό βρίσκεται εντός των ορίων του καμβά.

Στη συνέχεια, ξεκινούμε τη σάρωση των scanlines ξεκινώντας από το $y_{\min}$ έως το $y_{\max}$. Δεν περιλαμβάνουμε την τιμή $y_{\max}$ προς αποφυγή χρωματισμού κορυφών 2 φορές (έτσι κάθε κορυφή ανήκει μόνο στην κάτω ακμή και όχι και

στην πάνω, η πάνω κορυφή θα ανήκει στην επόμενη ακμή). Προσθέτουμε στην λίστα των ενεργών ακμών όσες ακμές για τις οποίες η ελάχιστη τεταγμένη είναι ίση με y (την τρέχουσα τιμή της εξεταζόμενης γραμμής σάρωσης) και αφαιρούμε αυτές που έχουν y_{max} ίσο με y . Η προκύπτουσα λίστα ενεργών ακμών για αυτή τη γραμμή σάρωσης ταξινομείται, δηλαδή ταξινομούνται οι τετράδες τιμών κάθε ακμής, κατά την τρέχουσα τετμημένη τομής της ακμής με το scanline (το 3^ο στοιχείο κάθε τετράδας).

Ανάμεσα σε κάθε ζεύγος σημείων τομής των ενεργών ακμών με το scanline είναι το διάστημα πλήρωσης. Το τρίγωνο είναι κυρτό πολύγωνο και έτσι σε κάθε scanline προκύπτουν το πολύ δύο ενεργές ακμές και συνεπώς δύο ενεργά σημεία που ορίζουν ένα συνεχόμενο διάστημα πλήρωσης. Ο αλγόριθμος δίνεται έτσι ως επέκταση του αλγορίθμου πλήρωσης πολυγώνων.

Για x από x_{start} (την τετμημένη του πρώτου ενεργού σημείου) έως $x = x_{end}$ (τετμημένη του τέλους του διαστήματος πλήρωσης, φροντίζουμε ώστε να περιλαμβάνεται στο διάστημα που διατρέχει η for) χρωματίζουμε όλα τα σημεία με συντεταγμένες (x, y) , όπου y είναι το τρέχον scanline. Το χρώμα υπολογίζεται ως ο μέσος όρος των χρωμάτων των 3 κορυφών του τριγώνου, είναι δηλαδή σταθερό για κάθε pixel.

Τέλος, κλείνουμε το βρόχο εξέτασης των γραμμών σάρωσης με ενημέρωση των x των ενεργών σημείων χρησιμοποιώντας το λόγο dx/dy της κάθε ενεργής ακμής. Δηλαδή, το ενεργό σημείο κινείται κατά μήκος της ενεργούς πλευράς όταν κινούμαστε στην επόμενη γραμμή σάρωσης.

Σημείωση: Στα x_{start} και x_{end} χρησιμοποιούνται οι συναρτήσεις της βιβλιοθήκης numpy ceil & floor αντίστοιχα. Αυτό γίνεται γιατί τα σημεία τομής των ενεργών πλευρών με το scanline δεν έχουν απαραίτητα ακέραιες συντεταγμένες. Με την επιλογή ceil & floor διασφαλίζουμε ότι για το x_{start} επιλέγεται η επόμενη ακέραια θέση και για το x_{end} η προηγούμενη. Έτσι, παίρνουμε σίγουρα σωστά τα σημεία που βρίσκονται εντός του τριγώνου. Τέλος, επειδή αυτές επιστρέφουν τιμές float, γίνονται cast σε integer.

Gouraud shading

Η λογική του αλγορίθμου είναι η ίδια με του flat shading. Αυτό που αλλάζει είναι ο τρόπος χρωματισμού των pixel του τριγώνου.

Σε αυτή την περίπτωση, έχουμε υλοποιήσει την drawpixel με ορίσματα (img, vertices, vcolors, p, area). Η drawpixel χρωματίζει το σημείο p (tuple ή list με τη μορφή (x, y)) χρησιμοποιώντας και την παράμετρο area. Αυτή υπολογίζεται από την συνάρτηση triangle_area(p1, p2, p3). Επιστρέφει, δηλαδή, το εμβαδόν του τριγώνου που ορίζεται από τις συντεταγμένες των 3 σημείων στα ορίσματα.

Η drawpixel υπολογίζει το χρώμα του pixel χρησιμοποιώντας τα βάρη w_0, w_1, w_2 . Για το σημείο $P=(x, y)$ εντός του τριγώνου που ορίζεται από τις κορυφές $P_0=(x_0, y_0)$, $P_1=(x_1, y_1)$, $P_2=(x_2, y_2)$ υπάρχουν βάρη w_0, w_1, w_2 , τέτοια ώστε:

$$w_0 + w_1 + w_2 = 1 \text{ και } w_0 P_0 + w_1 P_1 + w_2 P_2 = P.$$

Τα w_0, w_1, w_2 υπολογίζονται καταλλήλως μέσω των εμβαδών που ορίζονται από τριάδες των κορυφών P, P_0, P_1, P_2 .

Τότε το χρώμα c σημείο p δίνεται από:

$$c = w_0 c_0 + w_1 c_1 + w_2 c_2$$

όπου c_i , για $i = 0, 1, 2$ είναι το RGB χρώμα στο αντίστοιχο σημείο (διάνυσμα με τρεις χρωματικές συνιστώσες).

Shading με texture maps

Αρχικά ορίζεται η συνάρτηση `vector_interp1` (είναι και το πρώτο ζητούμενο της εργασίας). Σε αυτή λαμβάνεται υπόψη και η περίπτωση κατά την οποία στην παρεμβολή εμφανίζεται 0 στον παρονομαστή, όταν δηλαδή, $x_1 = x_2$ (κάθετη ακμή) για $\text{dim}=1$ και $y_1 = y_2$ (οριζόντια ακμή) για $\text{dim}=2$. Σε αυτή την περίπτωση απλά επιστρέφεται η τιμή της διανυσματικής συνάρτησης στο σημείο p_1 .

Η κύρια λογική του αλγορίθμου παραμένει η ίδια, αλλά τώρα στις τιμές που αποθηκεύουμε για κάθε ακμή στη λίστα `edges` προστίθενται τα `indices` των κορυφών που ορίζουν την ακμή. Ως πέμπτο στοιχείο προστίθεται το `index` της κορυφής με τη χαμηλότερη τεταγμένη από το κάθε ζεύγος και ως τελευταίο το `index` της άλλης κορυφής.

Ο λόγος είναι ότι θα χρειαστεί να γνωρίζουμε τις κορυφές που ορίζουν τις ενεργές πλευρές για να υπολογίζουμε τις κανονικοποιημένες συντεταγμένες στα σημεία A και B (τις άκρες του διαστήματος πλήρωσης) μέσω γραμμικής παρεμβολής (`vector_interp1`) χρησιμοποιώντας τις κανονικοποιημένες συντεταγμένες των κορυφών. Δηλαδή ανακτούμε τη λίστα των έξι τιμών για κάθε ακμή που αποθηκεύσαμε στο πρώτο βήμα, και από αυτές ανακτούμε τα `indices` των κορυφών που ορίζουν τις ενεργές ακμές, πάνω στις οποίες βρίσκονται τα σημεία A και B.

Έπειτα, κάνουμε γραμμική παρεμβολή ξανά, τώρα για κάθε σημείο του scanline μεταξύ των A και B. Χρησιμοποιούμε τις κανονικοποιημένες συντεταγμένες των A και B.

Επίσης, χρησιμοποιούμε τη συνάρτηση `clip` με `interval [0, 1]` ώστε οι τιμές των u & v που είναι μικρότερες από 0 να γίνονται 0, και όσες είναι μεγαλύτερες από 1 να γίνονται 1. Έτσι, επιβεβαιώνουμε ότι οι τιμές των u & v βρίσκονται εντός του εύρους `[0, 1]`.

Ο λόγος που λαμβάνουμε:

$u = \text{int}(u * (tw - 1))$ και $v = \text{int}(v * (th - 1))$

είναι ο εξής: Το πλήθος των pixels στην texture image είναι tw (στο πλάτος) και th (στο ύψος). Επομένως τα indices για αυτά θα παίρνουν τιμές από 0 έως $tw - 1$ και 0 έως $th - 1$. Εμείς θέλουμε να αντιστοιχίσουμε από τιμές στο $[0, 1]$ σε τιμές των indices $[0, tw - 1]$. Δηλαδή για $u=1$, αντιστοιχίζεται το $tw - 1$ πχ, και έτσι βρίσκόμαστε εντός ορίων. Το casting σε int γίνεται ώστε στην περίπτωση αντιγραφής χρώματος από μη ακέραια θέση να επιλέγεται η πλησιέστερη ακέραια τιμή.

render_img

Αρχικοποιούμε τις διαστάσεις του καμβά σε $M = 512$, $N = 512$ και το background της εικόνας σε λευκό.

Κάνουμε reshape το depths σε $(L,) = (128776,)$ για αποφυγή προβλημάτων αργότερα.

Μέσω του κώδικα:

```
triangle_depths = depth[faces].mean(axis=1)
```

υπολογίζουμε πρώτα τα depths των κορυφών που απαρτίζουν το κάθε τρίγωνο. Ο πίνακας faces περιέχει τους δείκτες των κορυφών (που αναφέρονται στον πίνακα vertices) που ορίζουν το κάθε τρίγωνο. Η i-οστή γραμμή περιέχει τους δείκτες των κορυφών του i-οστού τριγώνου. Διαβιβάζουμε τους δείκτες του πίνακα faces στον πίνακα depths (έχει ίδια διάσταση με τον πίνακα vertices), οπότε προκύπτει πίνακας ίδιας διάστασης με τον πίνακα faces $(K, 3) = (63488, 3)$. Κάθε γραμμή περιέχει πλέον το depth της κάθε κορυφής του τριγώνου, και σε αυτόν τον πίνακα υπολογίζουμε κατά μήκος των γραμμών τον μέσο όρο (όπως λένε οι οδηγίες).

```
indices = np.argsort(-triangle_depths):
```

- negating array: Τα μικρότερα στοιχεία γίνονται τα μεγαλύτερα και τούμπαλιν.
- Η argsort(...) επιστρέφει κανονικά τους δείκτες της λίστας του ορίσματος, αν αυτή ήταν ταξινομημένη από το μικρότερο προς το μεγαλύτερο στοιχείο. Σε συνδυασμό με το negating array παίρνουμε πλέον τους δείκτες των στοιχείων από τα μεγαλύτερα προς τα μικρότερα.

Τώρα, μπορούμε να διατρέξουμε τη λίστα των depths των τριγώνων ξεκινώντας από αυτά με το μεγαλύτερο depth με descending order.

```
for i in indices:  
    face = faces[i]
```

Πρώτα παίρνουμε τους δείκτες των κορυφών του i-οστού τριγώνου. Μετά παίρνουμε τις συντεταγμένες των κορυφών τους, το χρώμα τους, όπως και τις κανονικοποιημένες συντεταγμένες τους.

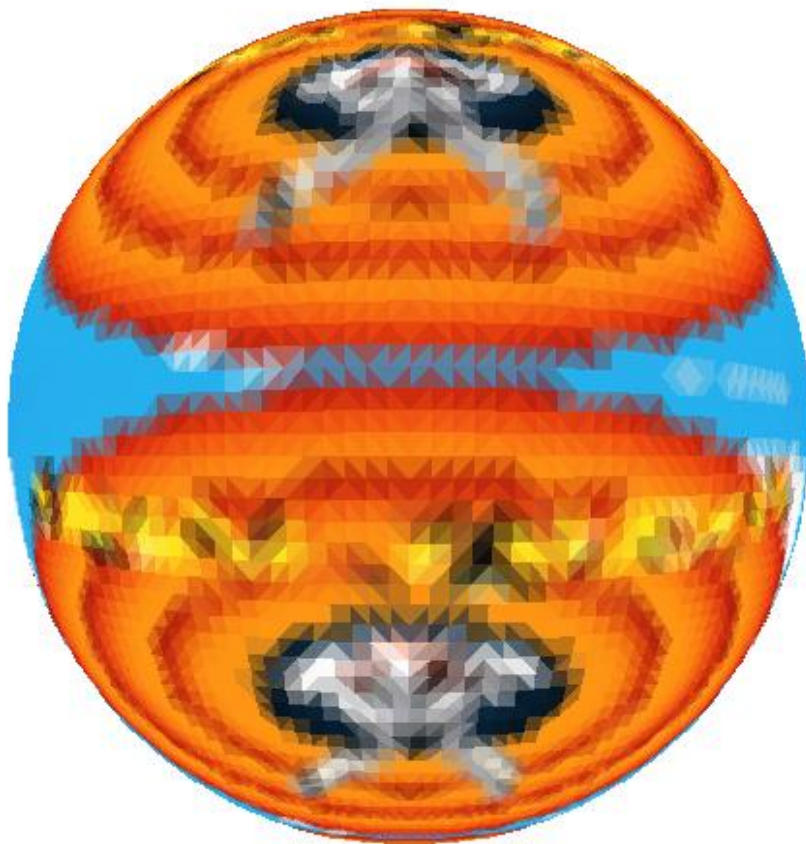
Με εντολές if, καλούμε την ανάλογη συνάρτηση χρωματισμού με τον σωστό τρόπο, και επιστρέφουμε την εικόνα.

Scripts

Στα scripts, πρώτα φορτώνουμε τα δεδομένα με τον συγκεκριμένο τρόπο. Προσοχή χρειάζεται στο ότι οι μέθοδοι της OpenCV χρησιμοποιούν BGR χρώμα, και για αυτό πρέπει να γίνεται σωστά μετατροπή όταν χρειάζεται.

Αποτελέσματα:

- **flat_shading**



- gouraud shading



- texture map shading

